ESP32 Setup

BDI310 Creative Coding · Week 6 · Follow this at your desk, one step at a time

1 Check your kit

Part	What it looks like
ESP32 board	A blue board with a small camera on it and a USB-C socket at one end
Breadboard	A white board covered in small holes
LED	A small coloured bulb with two legs
RGB LED	A clear or cloudy white bulb with four legs
Resistor	A tiny tube with coloured stripes and a wire out of each end. You have one.
Push button	A small square with a round button on top and four legs
Buzzer	A small black cylinder with two legs
Jumper wires	Coloured wires with a metal pin on each end
USB-C cable	Joins the board to your laptop

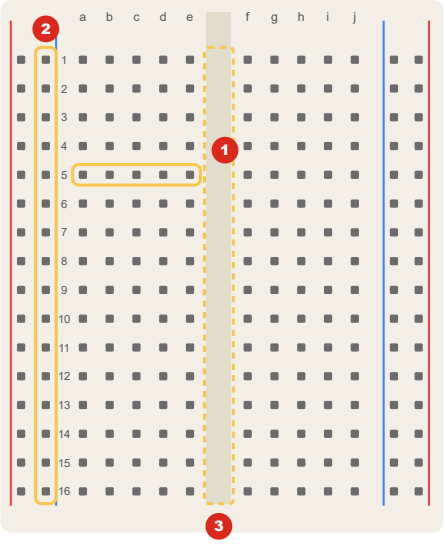
2 The board

- The **USB-C socket** at one end. The cable from your laptop powers the board and carries your code onto it.
- The **pins** along the long edges. Each has a number or name printed beside it, such as 13, 14, GND or 3V3. Code that says 13 means the pin with 13 printed beside it.
- **GND** pins are the way back. Power leaves the board from a pin, goes through your part, and comes back through a GND pin. If a part has no wire leading back to a GND pin, power cannot flow through it, and it stays off.
- A tiny light with **IO2** printed beside it. You switch it on first.
- A tiny button marked **BOOT** or **IO0**. You only press it if sending code to the board gets stuck.
- **Today, use only pins 2, 13, 14, 15, 32 and 33.** These six pins stay free even when the camera is switched on. Our code does not use the camera, but sticking to these six means you never have to think about it.

> **Pull the USB cable out every time you move a wire. Never join a 3V3 or 5V pin straight to a GND pin with a wire: it can destroy the board.**

Where the board gets its power. The USB cable is the board's only power. Cable in: the board has power and runs whatever code is on it. Cable out: the board has no power at all, and nothing on the breadboard is live. That is why you pull the cable out before you move a wire: with no power, a wire in the wrong hole cannot damage anything. Plug it back in when the wiring is done. Your code is still on the board and starts running again. Once the code is on, the board does not need the laptop, only power: a phone charger with a USB-C cable works too.

3 The breadboard



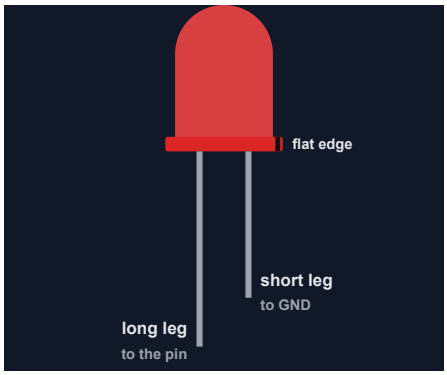
1. Each numbered row has **five holes on each side** of the groove. The five holes in one row, on one side, are joined to each other.
2. Along each long edge is a line of holes beside a **blue stripe**. Every hole in that line is joined, top to bottom. We call the one on the left **the GND line**. One wire from a GND pin to it, and every part can reach GND from there.
3. The **groove** down the middle. Holes on the left of it are not joined to holes on the right.

Two legs in the same row, on the same side of the groove, are joined. Two legs in different rows are not.

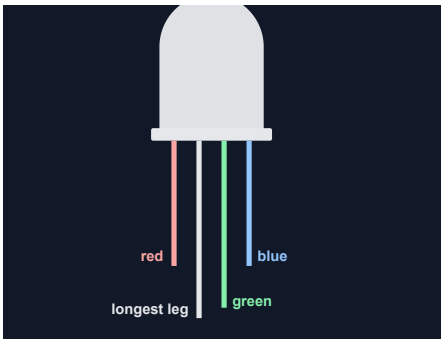
The board is wide and covers most of a breadboard. Put your parts on the free part, or on a second breadboard, and use jumper wires from the board's pins to your parts' rows.

4 The parts, and which way round they go

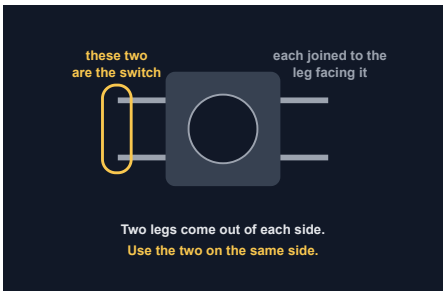
Polarity: some parts only work one way round. The LED, the RGB LED and the buzzer each have a + leg and a - leg. Power has to go in through the + leg and out through the - leg, so the + leg goes towards the pin and the - leg towards GND. The legs are different lengths so you can tell them apart. The resistor, the push button and the jumper wires have no polarity: they work either way round. A part with polarity put in the wrong way round does not work, and the fix is to turn it round.



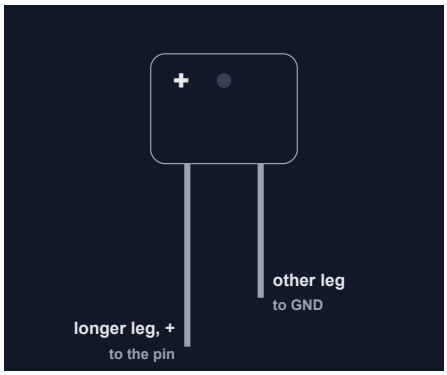
LED. Has polarity. The long leg is the + leg and connects to the pin. The short leg is the - leg and goes towards GND. If you put it in backwards, it stays dark but is not damaged. Turn it round and it lights up.



RGB LED. Hold it with the longest leg second from the left. Left to right: red, longest (shared, the resistor goes here), green, blue.



Push button. Turn it over: two legs come out of one side, two out of the opposite side. The two on the same side are the switch: they only join while you press. Each leg is always joined to the one facing it.



Buzzer. Has polarity. The longer leg, or the one under the +, is the + leg and connects to the pin. The other leg is the - leg and goes to GND.



Resistor. Holds back some power so a light does not burn out. Either way round. It goes with whichever light you are using.

5 Setting up the device

Your laptop needs three things to send code to the board. Each one comes from the people who make it, so you know where to go back to.

What	What it does	Where it comes from
PlatformIO	An add-on for VS Code. It sends code down the USB cable onto the board.	Inside VS Code. Its makers' site is platformio.org.

The ESP32 package, called Espressif 32	Teaches PlatformIO how to build code for an ESP32. Espressif is the company that makes the ESP32.	Inside PlatformIO.
The USB driver, called CH340	A driver is a small program that tells your laptop how to talk to a device. This one lets your laptop recognise the board.	WCH, the company that makes the board's USB chip.

1. **Open VS Code.** If it is not on your laptop, download it from code.visualstudio.com and install it.
2. **Install PlatformIO.** Down the left edge of VS Code is a column of icons. Click the one made of four small squares, **Extensions**. Type **PlatformIO IDE** in the search box, click it in the list, then click **Install**. Wait while it downloads, when it asks you to reload VS Code, click Reload. An icon shaped like an alien head appears down the left edge.
3. **Open PlatformIO Home.** Click the alien head. Under **Quick Access**, click **PIO Home**, then **Open**. A page called PIO Home opens, with a menu down its left side: Home, Projects, Libraries, Boards, Platforms, Devices.
4. **Install the ESP32 package.** In the PIO Home menu, click **Platforms**, then the **Embedded** tab. Type **Espressif 32** in the search box, click it in the list, then click **Install**. It is a large download: wait until it says it is installed.
5. **Install the USB driver.**
   - **Windows:** go to WCH's download page, wch-ic.com/downloads/CH341SER_EXE.html, and click **Download**. The file is called **CH341SER.EXE**: the driver is named after a sister chip, CH341, and covers your CH340 too. Open the file, click **Yes** if Windows asks to let it make changes, then click **INSTALL**. Wait for the message that it installed, then close the window.
   - **Mac:** newer Macs already have a driver for this chip, so check first with the next step. Only if your board does not show up, go to WCH's download page, wch-ic.com/downloads/CH34XSER_MAC_ZIP.html, and click **Download**. Open the zip, open the installer inside it, and follow its steps. Then open **System Settings**, **Privacy & Security**: if it says software from WCH was blocked, click **Allow**. Restart the Mac.
6. **Plug the board in and check your laptop can see it.** Plug the USB-C cable into the board and your laptop. A small light on the board comes on. In PIO Home, click **Devices**, then **Refresh**. On Windows you should see a line with **USB-SERIAL CH340** and COM and a number, such as COM5. On a Mac you should see a line with **wchusbserial** or **usbserial** in it. That line is your board.
7. **Board not in the list?** Try a different USB cable first: some cables can only charge a phone and cannot carry code. Then try a different USB socket on your laptop. Then install the driver again, pull the cable out, plug it back in, and click Refresh.
8. **Download the starter projects and open the first one.** Go to github.com/tolumebaanne/esp32-starter, click the green **Code** button, then **Download ZIP**. The same zip is also on Blackboard. Unzip it. Inside are six folders, 01 to 06, one for each part of today. In VS Code, click **File**, **Open Folder**, and choose the folder **01_blink_onboard** itself, not the folder around it. If VS Code asks whether you trust the authors, click Yes. The code is in the **src** folder, in a file called **main.cpp**.

6 Make the board's light blink

Nothing to wire. Project 01 is already open.

What is in a project folder. Every numbered folder looks the same:

File or folder	What it is
`src/main.cpp`	The code. The only file you open and change.
`platformio.ini`	Tells PlatformIO which board this is. Leave it alone.
`.pio/` and `.vscode/`	Appear by themselves after your first upload. Leave them alone.

The language. The board's code is written in **C++**, the Arduino way: words such as setup, loop and digitalWrite come from Arduino, a set of ready-made words for boards like this one. It is not JavaScript, because JavaScript runs inside a web browser, and the board has no browser and no screen. C++ is turned into instructions the chip runs directly, so it is small and fast, and it is what the ESP32's makers and nearly every sensor example use, so an LLM has seen a lot of it. Your web page stays in JavaScript.

The skeleton. Every program for this board starts with the same three parts:

```
#include <Arduino.h>    // brings in the board's own words, such as digitalWrite and delay

void setup() {
  // runs once, the moment the board gets power
}

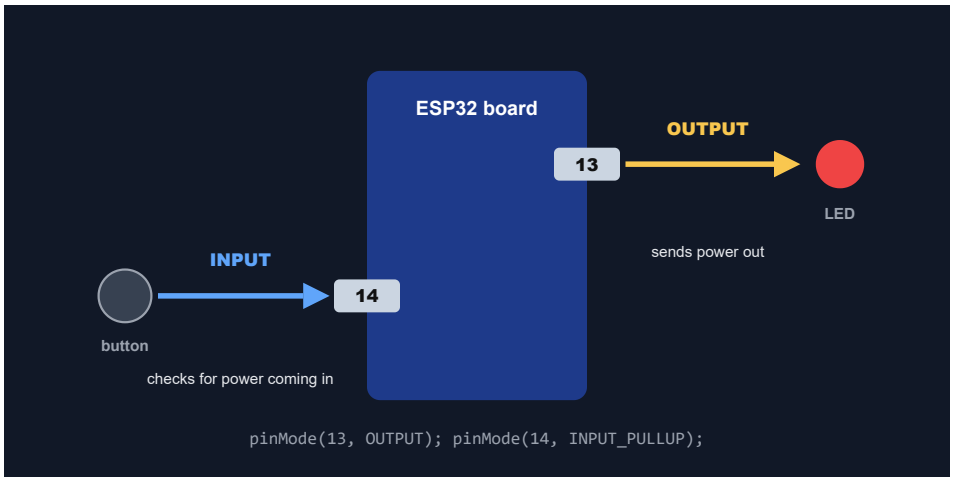
void loop() {
  // runs again and again until the power goes
}
```

void means the function hands nothing back when it finishes. In Week 4 a function could return something, the way decide() handed back a message. setup and loop do their job and return nothing.

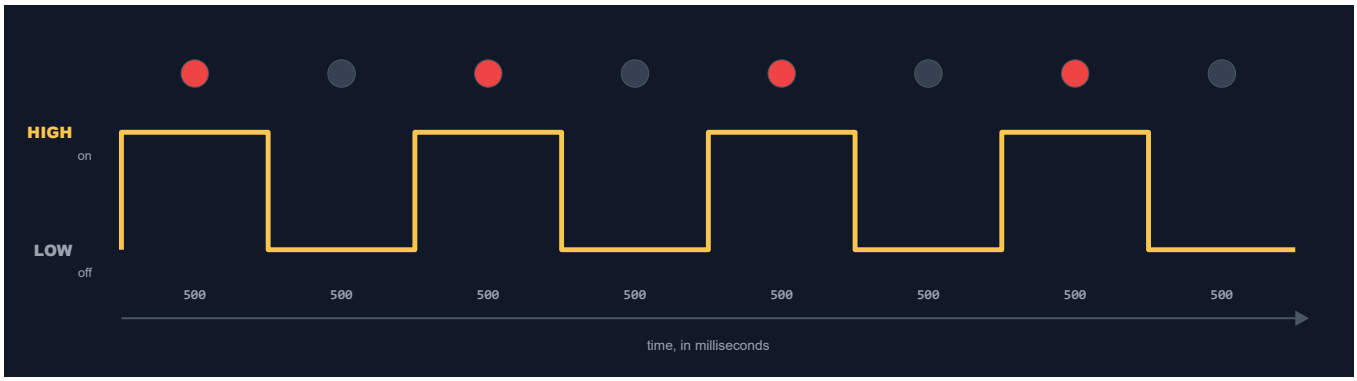
The Week 4 words, written for the board. The board's code says what kind of value goes in a box: `int` for a whole number, `bool` for true or false. Semicolons, curly brackets and comments are the same as JavaScript.

Idea	In JavaScript	In the board's code
Constant	`const COLD = 18;`	`const int COLD = 18;`
Variable	`let count = 0;`	`int count = 0;`
True or false	`let pressed = true;`	`bool pressed = true;`
Function	`function show() { }`	`void show() { }`
Conditional	`if (count > 3) { }`	`if (count > 3) { }`
Comment	`// a note`	`// a note`

A pin works one of two ways. As an **output**, it sends power out: that is how the board switches the LED and the buzzer on. As an **input**, it checks whether power is coming in: that is how the board reads the button. `pinMode` in setup tells each pin which way to work.



A pin is either HIGH or LOW. HIGH means power is on, LOW means power is off. HIGH is a 1 and LOW is a 0. The blink code switches pin 2 HIGH, waits, switches it LOW, waits:



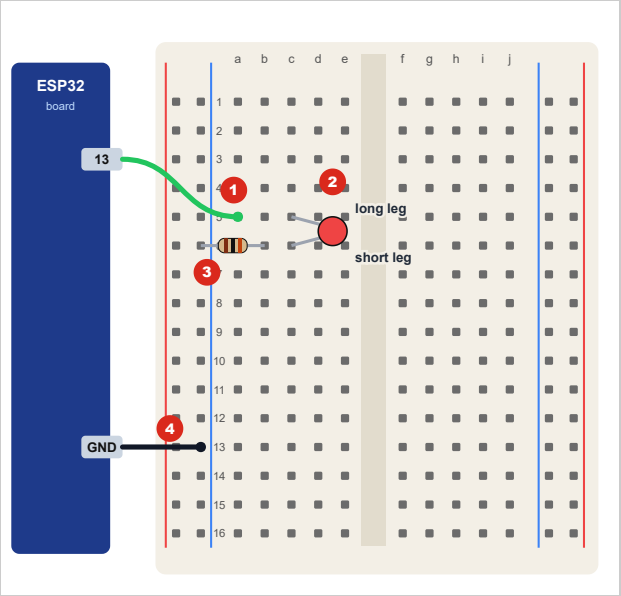
Now upload it.

1. Along the bottom of VS Code is a coloured bar with small icons. Click the **right arrow**. That is Upload. The first upload takes a few minutes, later ones are quicker. Wait until the text says **SUCCESS**. The light beside IO2 blinks.
2. If the text stops at **Connecting...**, click the right arrow again and hold the **BOOT** button while it says Connecting. Let go when it shows percentages.

Three tasks. For each one: change the code, save it, upload it, and watch the board. Try it before you look at the slide that explains it.

1. **Task 1.** Make the light blink faster.
2. **Task 2.** Make the light stay on longer than it stays off.
3. **Task 3.** Make the light blink twice, quickly, then pause before it starts again.

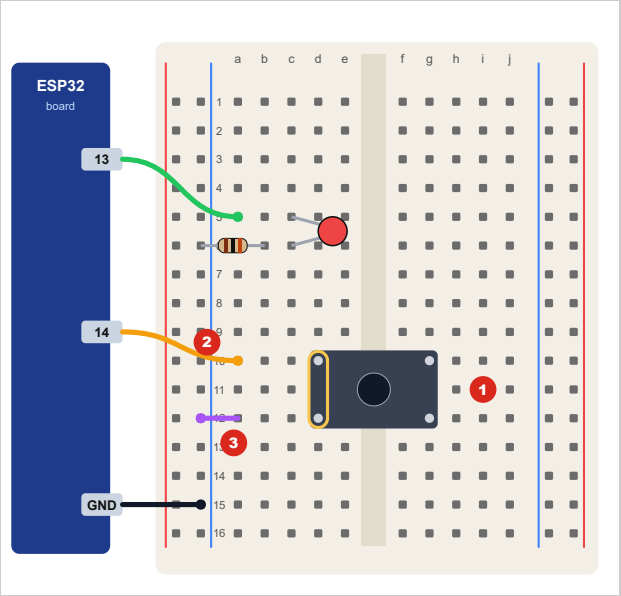
7 Wire an LED and make it blink



1. Pull the USB cable out. A jumper wire from **pin 13** to a hole in **row 5**.
2. The LED's **long leg in row 5** and its **short leg in row 6**, on the same side of the groove as the wire.
3. The resistor from **row 6** to the **GND line**.
4. A jumper wire from a **GND pin** to the GND line.

Plug the cable back in. Open the folder **02_blink_led** and click the right arrow. The LED blinks. If it stays dark, pull the cable out and turn the LED round.

8 Add a button that turns the LED on

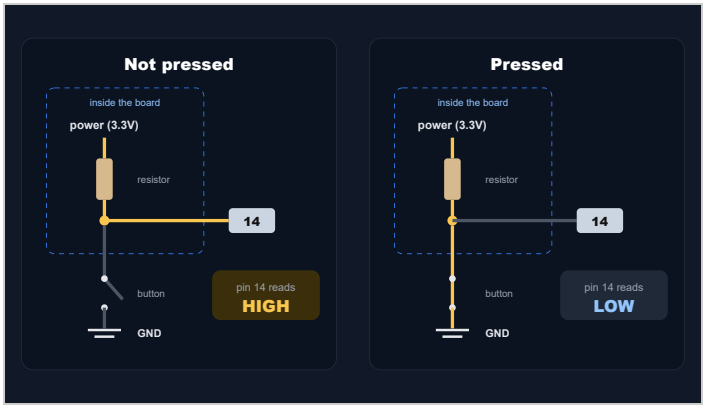


1. Pull the cable out. Keep the LED. Sit the button **across the groove**, two legs on the left of it and two on the right, in **rows 10 and 12**.
2. A jumper wire from **pin 14** to **row 10**, on the left of the groove.
3. A jumper wire from **row 12**, on the left of the groove, to the **GND line**. The two legs on the right stay unused.

No resistor needed: the board has one built in for buttons. Open the folder **03_button** and upload it. Press: LED on. Let go: LED off. LED on all the time? The button is not across the groove. Turn it a quarter turn.

In the bottom bar, click the icon shaped like a **plug**. That is the Serial Monitor. The board tells you what it reads: pressed, or not pressed.

Why a press reads LOW. An input pin with nothing joined to it gives random readings. `INPUT_PULLUP` turns on a resistor inside the board that joins pin 14 to power, so not pressed, the pin reads HIGH. Pressing the button joins pin 14 to GND, so pressed, it reads LOW. This is called a **pull-up**.

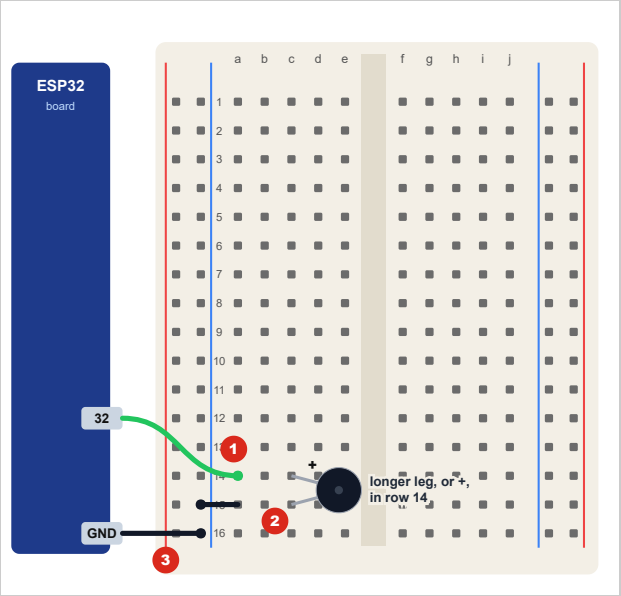


One step further: one press, three blinks. In `main.cpp`, scroll down to `loop` at the bottom. Put `//` in front of `holdToLight();` and take the `//` off `threeBlinks();`. A line starting with `//` is a comment, so the board skips it. Save, upload, and press the button once. Only one of the two lines should be without `//`.

Hold to light is a **condition**: the board checks the button all the time and the LED follows it. **Three blinks**

is a **trigger**: the press happened, so the board does one job straight away and then waits for the next press.

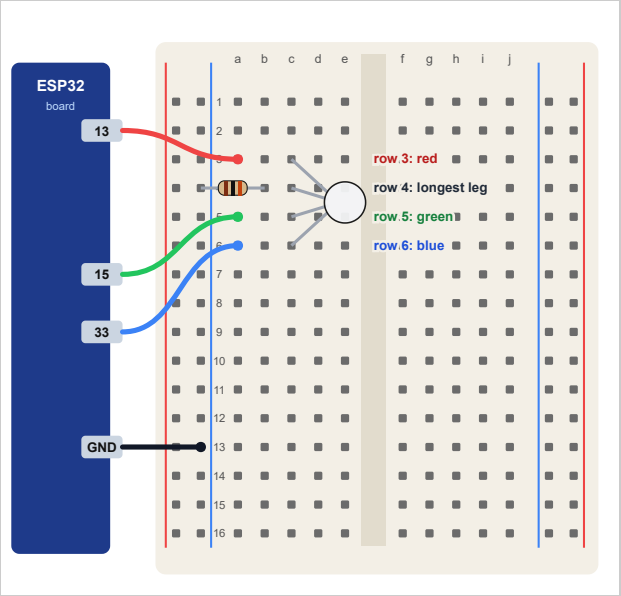
9 Add a buzzer and find out which kind it is



1. Pull the cable out. Leave the LED and button where they are. The buzzer's legs in **rows 14 and 15**, longer leg in row 14.
2. A jumper wire from **pin 32** to **row 14**.
3. A jumper wire from **row 15** to the **GND line**.

Open the folder **04_buzzer**, upload it, and listen to the first second. **A long beep:** you have an active buzzer, which makes its own sound. **Only a click:** you have a passive buzzer, which needs the code to play a note. Then it plays three notes. Both kinds play them.

10 Swap in the RGB LED



1. Pull the cable out. Take the LED and the resistor out. You have one resistor and the RGB LED needs it now. Leave the button and buzzer.
2. The four legs in **rows 3 to 6**, longest leg in row 4: red in 3, longest in 4, green in 5, blue in 6.
3. The resistor from **row 4** to the **GND line**.
4. Jumper wires: **pin 13** to row 3, **pin 15** to row 5, **pin 33** to row 6. The wire from pin 13 used to go to the LED; move its breadboard end to row 3.

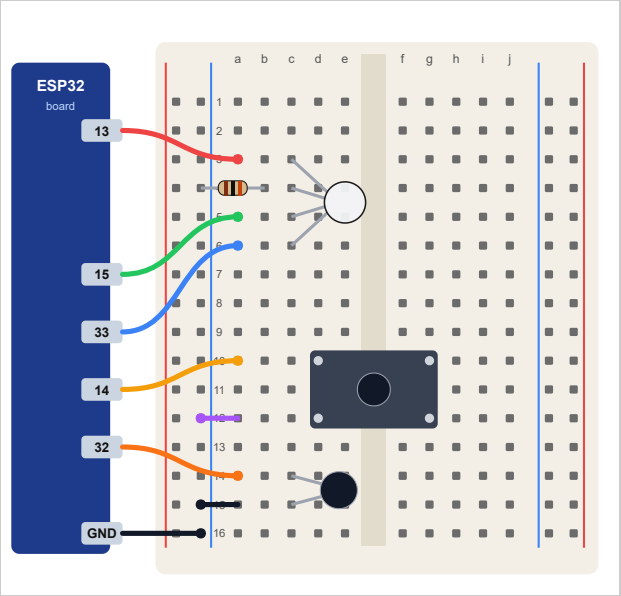
Open the folder **05_rgb** and upload it. It goes red, green, blue, off, and starts again.

Nothing lights at all? Yours is the other kind. Pull the cable out. Take the resistor's end out of the GND line, push it into an empty row, and run a jumper wire from that row to a **3V3** pin. In the code, change `false` to `true` on the `COMMON_ANODE` line. Save, plug in, and upload again.

Colours in the wrong order? Swap the jumper wires between the legs.

One step further: a traffic light. In `loop` at the bottom of `main.cpp`, put `//` in front of `cycleColours();` and take the `//` off `trafficLight();`. Save and upload. It goes green, then yellow, then red. The RGB LED has no yellow light: red and green switched on together make yellow. With one resistor shared by both, it may look more orange than yellow.

11 Press the button to change colour



1. Keep everything wired. Open the folder **06_all_together**.
2. If you changed COMMON_ANODE to true in step 10, change it to true here too.
3. Upload. Each press of the button changes the colour and beeps.

Input: the button. **State:** which colour it is on. **Output:** the colour. **Feedback:** the beep, which tells you the board noticed the press.

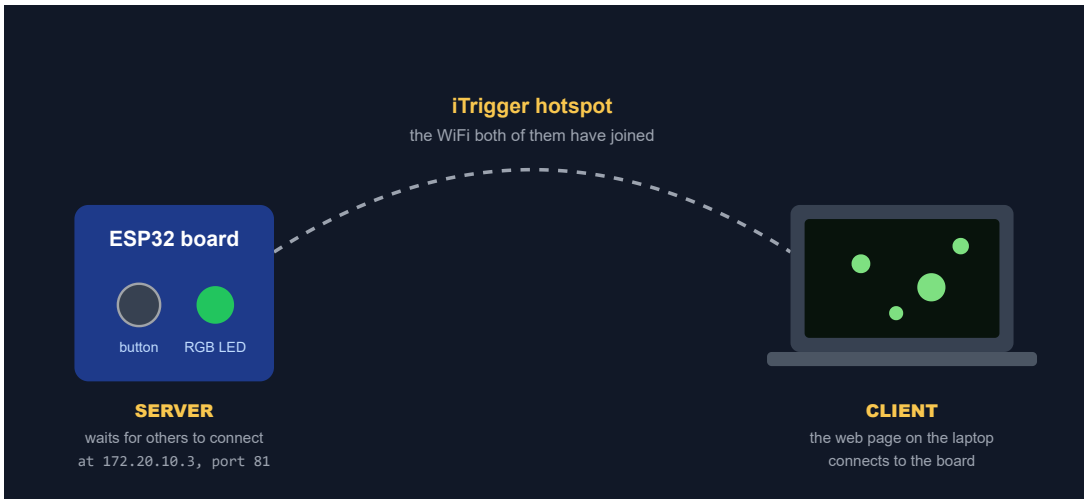
12 Button art

A button on the desk paints a picture on the projector. The board runs on a power bank and talks to the laptop over WiFi.

- **One press:** a bloom of shapes bursts open on the screen.
- **Two presses:** all the colours change, and the RGB LED changes to match the screen.
- **Three presses:** a wave sweeps across and wipes the screen clean.



How it works.



1. The board joins the hotspot and gets an address on it, such as 172.20.10.3.
2. The board runs a **server**: a program that waits for other programs to connect to it.
3. The web page is the **client**. Clicking Connect opens a **live connection** to the board and keeps it open.
4. The board counts your presses and sends one message down the connection: `press:1`, `press:2` or `press:3`.
5. The page reads the message and runs the matching code, which redraws the screen about 60 times a second.
6. After new colours, the page sends one back, such as `colour:1,0,0`, and the board sets the RGB LED to match.

Two programs, C++ on the board and JavaScript in the page, talking over one connection.

The four words. Input: the button presses. State: how many presses so far, and which colours the page is using. Output: the picture on the screen and the colour of the light. Feedback: the beeps, and the light blinking once for each press.

This is interactive media. A person does something with their hands, and a picture answers them. Everything you wired today, and all the code, is the part in between.

For the volunteer.

1. Pull the USB cable out, then check every wire against the drawing in step 11: the RGB LED on pins 13, 15 and 33 with its resistor to the GND line, the button on pin 14 with its other left leg to the GND line, the buzzer on pin 32 with its other leg to the GND line, and a GND pin to the GND line.
2. In VS Code, click **File**, **Open Folder**, and choose **07_web_art**.
3. If you changed COMMON_ANODE to true for your RGB LED, change it to true here too.
4. Plug the board in and upload. The first upload downloads one extra part for the WiFi connection, so it takes a little longer.
5. Click the **plug** icon in the bottom bar to open the Serial Monitor. Write down the number after **The board's address is**, such as 172.20.10.3.
6. Pull the cable out of the laptop and plug it into the power bank. The RGB LED blinks blue while the board joins the WiFi.
7. On the laptop at the projector, which is on the same WiFi, the page is open. Click **Connect**, and the dot turns green. If it stays red, type the number you wrote down into the box and click Connect again.

When it does not work

What you see	What to check
Upload cannot find the board	Try a different USB cable. Then install the CH340 driver.
Stuck on Connecting...	Click the right arrow again and hold BOOT while it says Connecting.
No arrow in the bottom bar	Open the project with File, Open Folder, choosing the numbered folder itself.
The LED stays dark	Turn it round. Then check the wire is in the pin the code names.
Nothing works at all	Check there is a wire from a GND pin to the GND line.
Parts low down do nothing	Some breadboards split the GND line halfway down. Move the part up, or add a wire across the split.
The LED is always on	The button is not across the groove. Turn it a quarter turn.

Every pin you used

Pin	What is on it
2	The light on the board, beside IO2
13	The LED, and then the RGB LED's red leg
14	The push button
15	The RGB LED's green leg
32	The buzzer
33	The RGB LED's blue leg
GND	The way back for every part, through the GND line